6. Project Testing Phase

6.1 Introduction

Testing verifies that PocketSmart AI satisfies the specified requirements and performs its functions correctly. It identifies errors, security issues, integration problems, and unexpected behavior. Each module is tested individually and then as an integrated system.

6.2 Objectives of Testing

- To verify that all requirements are implemented correctly.
- To identify and correct software errors.
- To ensure authentication works properly and protected data is inaccessible to unauthorized users.
- To verify that transactions, budgets, and goals are stored and retrieved correctly.
- To verify correct calculations for totals, savings, and dashboard statistics.
- To evaluate the quality and relevance of Gemini-generated advice.
- To verify usability and reliability.

6.3 Testing Strategy

Unit Testing → Integration Testing → System Testing → User Acceptance Testing

6.4 Unit Testing

Individual functions are tested in isolation, including authentication functions, expense totals, budget calculations, prompt-building functions, response parsing, data validation, and database operations. For example, the monthly saving required for a goal is calculated as Required Monthly Saving = (Target Amount – Saved Amount) ÷ Months Remaining, and the budget utilization as Utilization = (Actual Spending ÷ Budgeted Amount) × 100. Different input combinations, including zero and negative values, are tested to confirm the expected result.

6.5 Integration Testing

- Frontend and backend communication.
- Backend and database operations.
- Authentication and protected APIs.
- Backend and Gemini API.
- Transaction module and dashboard statistics.
- Budget module and actual-vs-planned comparison.

6.6 System Testing

Complete workflows are tested: registration, login, adding income and expenses, generating a budget, viewing insights, creating a savings goal, requesting a recommendation, using the finance

chat, viewing history, viewing the dashboard, and logging out.

6.7 Functional Test Cases

Test ID	Function	Test Input / Action	Expected Result
FT01	Registration	Enter valid user details	Account is created
FT02	Registration	Enter incomplete details	Validation message appears
FT03	Login	Enter valid credentials	User is logged in
FT04	Login	Enter incorrect password	Login is rejected
FT05	Transactions	Add a valid expense	Entry is saved and listed
FT06	Transactions	Enter a negative or empty amount	Validation message appears
FT07	Categorization	Enter description "auto fare"	A suitable category is suggested
FT08	Budget Planner	Enter income, fixed costs, and goals	Structured category-wise budget is generated
FT09	Insights	Open insights with recorded expenses	Relevant spending summary is displayed
FT10	Savings Goal	Enter target amount and date	Correct monthly saving is shown
FT11	Recommendation	Enter need and budget limit	Options within the limit are shown with reasons
FT12	Finance Chat	Ask a follow-up question	Answer reflects earlier context
FT13	History	Open history page	Previous items are displayed
FT14	Dashboard	Open dashboard	Correct totals and charts appear
FT15	Logout	Select logout	Session ends

6.8 Authentication and Security Testing

- Access to protected APIs without a token is rejected.
- Expired or invalid tokens are rejected.
- A user cannot view or edit another user's transactions or goals.
- Passwords are stored as hashes.
- The Gemini API key is not exposed to the browser.

6.9 AI Response Testing

Gemini responses are evaluated for relevance, clarity, practicality, and format using sample scenarios such as low-income student budgets, family budgets, and savings plans. Cases tested include empty or very long input, unrealistic budgets, vague purchase needs, and malformed JSON output. Because AI-generated content can occasionally be inaccurate or unsuitable, the application displays a disclaimer that it is general guidance and advises users to verify important financial decisions.

6.10 Performance and Usability Testing

Page load time, API response time, and behavior during Gemini delays are observed. Usability is checked for navigation, clarity of messages, and responsiveness on desktop and mobile screens.

6.11 User Acceptance Testing

A small group of students and young professionals uses the application and provides feedback on ease of use, usefulness of insights, and quality of budgets and recommendations. Reported issues are corrected before the final demonstration.

6.12 Testing Summary

After testing and correction of defects, all major modules were found to operate according to the requirements, and the application was considered ready for demonstration.